

Large Language Model Meets Graph Neural Network in Knowledge Distillation

Shengxiang Hu
School of Computer Engineering and
Science, Shanghai University
Shanghai, China
shengxianghu@shu.edu.cn

Guobing Zou*
School of Computer Engineering and
Science, Shanghai University
Shanghai, China
gbzou@shu.edu.cn

Song Yang
School of Computer Engineering and
Science, Shanghai University
Shanghai, China
yangsong@shu.edu.cn

Yanlan Gan
School of Computer Science and
Technology, Donghua University
Shanghai, China
ylgan@dhu.edu.cn

Bofeng Zhang
School of Computer and Information
Engineering, Shanghai Polytechnic
University
Shanghai, China
bfzhang@sspu.edu.cn

Yixin Chen
Department of Computer Science and
Engineering, Washington University in
St. Louis
MO, USA
chen@cse.wustl.edu

ABSTRACT

Recent advancements in leveraging Large Language Models (LLMs) for Text-Attributed Graphs (TAGs) learning have shown significant potential, but practical deployment is often hindered by substantial computational and storage demands. Conventional Graph Neural Networks (GNNs) are more efficient but struggle with the intricate semantics embedded in TAGs. To combine the semantic understanding of LLMs with the efficiency of GNNs, we propose a novel LLM-to-GNN knowledge distillation framework, **Linguistic Graph Knowledge Distillation (LinguGKD)**, which employs TAG-oriented instruction tuning to train pre-trained LLMs as teachers and introduces a layer-adaptive contrastive distillation strategy to align node features between teacher LLMs and student GNNs within a latent space, effectively transferring the semantic and complex relational understanding from LLMs to GNNs. Extensive experiments across various LLM and GNN architectures on multiple datasets demonstrate that LinguGKD significantly enhances the predictive accuracy and convergence rate of GNNs without requiring additional training data or model parameters. Compared to teacher LLMs, the distilled GNNs offer superior inference speed and reduced resource requirements, making them highly practical for deployment in resource-constrained environments. Furthermore, our framework demonstrates significant potential for leveraging ongoing advancements in LLM research to continuously improve GNN performance.

1 INTRODUCTION

Text-Attributed Graphs (TAGs) integrate structured graph data with rich textual information, providing a comprehensive representation of complex systems and encapsulating extensive knowledge. These graphs are extensively utilized across diverse domains [29, 54]. Recently, the revolutionary impact of Large Language Models (LLMs), such as ChatGPT [34] and Llama [41], on natural language processing brings new opportunities and challenges to the application and research of TAGs.

While LLMs exhibit remarkable reasoning and problem-solving capabilities for complex tasks, they are not always satisfactory in every context. Studies [30, 35] have shown that integrating knowledge graphs can enhance the reasoning and handling capabilities of LLMs.

Furthermore, traditional Graph Neural Networks (GNNs) [5, 43, 47] excel at interpreting graph structures but struggle with semantic processing [27], especially as the complexity and volume of associated textual data increase. LLMs, with their exceptional contextual and relational understanding, offer a novel perspective in evaluating TAGs by effectively capturing the semantic nuances embedded in textual data. Integrating LLMs with GNNs addresses the semantic gap, leveraging the structural strengths of GNNs and the semantic prowess of LLMs. These scenarios highlight the demand for and challenges in researching LLM-based graph learning approaches, which can not only amplify semantic interpretation in TAGs but also enhance the overall performance of graph-based tasks, underscoring the necessity and significance of this research direction.

Recent advancements in LLM-based graph learning focus on two main approaches: LLM as Enhancer (LaE) and LLM as Predictor (LaP) [28]. The LaE approaches [6, 16, 45] enhance node embeddings in GNNs by utilizing the semantic processing capabilities of LLMs, addressing traditional GNN limitations in extracting semantic features from TAGs. In contrast, the LaP approaches [12, 44, 56] employ LLMs directly for prediction tasks within graph-related contexts, either by adapting Transformer-based models to incorporate graph structures or by encoding graph data in natural language for inference, thus significantly improving both semantic processing and structural understanding in graph-related tasks. These works demonstrate the potential of LLM as a foundational model for graph learning. However, the practical application of LLMs in graph learning faces significant challenges, particularly due to their large parameter sizes, often exceeding billions¹. This leads to high computational and storage demands, making cost-effective and widespread deployment challenging. Additionally, LLMs' extended latency during inference presents practical limitations in operational settings.

We consider whether it is possible to leverage the powerful semantic and entity relationship understanding capabilities of graph-oriented LLMs while reducing their computational and storage resource consumption, making them suitable for large-scale deployment in resource-constrained production environments. Addressing these challenges, knowledge distillation (KD) [4, 23, 37] from LLMs

¹https://huggingface.co/spaces/HuggingFaceH4/open_llm_leaderboard

to GNNs emerges as a promising strategy, enabling the transfer of LLM insights to more compact GNNs. This approach not only aims to reduce the model size and computational demands of LLMs but also leverages GNNs’ strengths in processing structured graph data. By distilling the semantic and structural understanding of LLMs into lightweight GNNs, it is expected to optimize graph reasoning tasks, facilitating more effective and adaptable real-world applications. However, LLMs and GNNs are designed for different types of data with significantly different model architectures, posing substantial challenges for knowledge transfer from LLMs to GNNs. This challenge remains largely unexplored in current research. Thus, developing effective methods for knowledge distillation between these models is crucial to unlocking their combined potential.

Under the above motivations, we propose a novel LLM-to-GNN knowledge distillation framework, **Linguistic Graph Knowledge Distillation (LinguGKD)**. To the best of our knowledge, this is the first framework that directly distills knowledge from teacher LLMs to student GNNs. Given the very early stage of graph-oriented LLM research and the lack of off-the-shelf LLMs designed specifically for graph tasks, inspired by [56], we begin by instruction tuning a pre-trained LLM (PLM) with carefully designed, tailored graph instruction prompts. This process equips the PLM with the capability to understand and process graph structures, thus obtaining an effective teacher LLM, named **LinguGraph LLM**. Then we introduce a layer-adaptive contrastive distillation strategy, complemented by a feature alignment mechanism to synchronize the feature spaces of the LLM and GNN, ensuring that the hierarchical node features learned by the teacher LLM are effectively aligned with those extracted by the student GNN. By doing so, it can propagate the teacher LLM’s deep semantic knowledge and intricate understanding of graph structures to the student GNN, leading to better TAG understanding capabilities.

Our extensive experimental evaluations, focused on node classification tasks, span various LLM and GNN models as well as multiple benchmark datasets, demonstrating the efficacy of the proposed LinguGKD framework in distilling graph knowledge from teacher LLMs to student GNNs and verifying its strong generality, suitable for different architectures of LLM and GNN. Specifically, the distilled GNNs exhibit a significant reduction in model complexity, making them more suitable for real-world applications with a much lower parameter count compared to LLMs, and a notable increase in inference speed. From the perspective of effectiveness, distilled GNNs not only achieve higher accuracy and faster convergence rates than the vanilla versions but also outperform those with advanced designs in certain scenarios, all without the need for additional training data or architectural changes. These results validate the effectiveness of our knowledge distillation framework and underscore its potential to enhance the practicality of LLMs in graph data processing, achieving an optimal balance between performance and efficiency.

The main contributions of this paper are summarized as follows:

- We conceptualize the novel research problem of knowledge distillation from LLMs to GNNs and propose an innovative graph knowledge distillation framework termed **LinguGKD**, which leverages the comprehensive semantic insights of graph-oriented teacher LLMs to enrich student

GNNs’ structural learning capabilities while maintaining their high efficiency.

- Within the LinguGKD framework, we develop a unique layer-adaptive contrastive distillation strategy, which ensures effective synchronization of hierarchical node features between the teacher LLM and the student GNN, guaranteeing the transfer of deep semantic knowledge and complex graph structural understanding.
- Extensive experimental evaluations across diverse LLM and GNN models as well as multiple benchmark datasets demonstrate that LinguGKD significantly enhances the classification accuracy of student GNNs while maintaining the lightweight nature. The distilled GNNs strike an optimal balance between performance in downstream tasks and efficiency in terms of time and space, making them practical for deployment on user devices in real-world scenarios.

The remainder of this paper is organized as follows: Section 2 provides the foundations and formulates the research problem; Section 3 delves into the details of the LinguGKD framework; Section 4 presents extensive experimental results and analyses; Section 5 reviews relevant literature; and Section 6 concludes the paper.

2 PRELIMINARIES

This section formalizes the key concepts central to our study.

DEFINITION 1 (TEXT-ATTRIBUTED GRAPH). A *Text-Attributed Graph (TAG)* is a graph where each node is associated with textual data. Formally, a TAG is denoted as $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{X})$, where $\mathcal{V} = \{v_i\}_{i=1}^n$ represents the set of nodes, with n being the total number of nodes, $\mathcal{E}$ is the set of edges with $e_{ij} \in \mathcal{E}$ indicating an edge between nodes v_i and v_j , and $\mathcal{X} = \{x_i\}_{i=1}^n$ denotes the node attributes, where x_i represents the textual attribute of node v_i .

Given a TAG $\mathcal{G}$, GNNs are essential for interpreting the graph’s topological dependencies:

DEFINITION 2 (GRAPH NEURAL NETWORK). *Graph Neural Networks (GNNs)* are specialized for handling graph-structured data, primarily through a k -layer message-passing mechanism [24], enabling the capture and analysis of k -hop node relationships and graph dynamics. This process is defined as:

$$\mathbf{h}_v^{(k)} = f(\mathbf{h}_v^{(k-1)}, \bigoplus_{u \in \mathcal{N}(v)} g(\mathbf{h}_u^{(k-1)}, \mathbf{h}_v^{(k-1)})) \quad (1)$$

where $\mathbf{h}_v^{(k)}$ is the feature representation of node v at the k -th layer, $\mathcal{N}(v)$ includes v ’s neighboring nodes, and functions $g(\cdot)$ and $f(\cdot)$ are trainable, responsible for aggregating neighbor features and updating node features, respectively. The operator $\bigoplus$ denotes an aggregation function, such as summation [60] or averaging [24].

While GNNs excel at processing graph structures, they have limitations in understanding individual node semantics. To leverage the capability of LLMs in semantic feature learning, an LLM-to-GNN Graph Knowledge Distillation framework can be designed to distill knowledge from a teacher LLM to a student GNN, enhancing the GNN’s capability in interpreting node semantics and complex graph topology. This can be defined as follows:

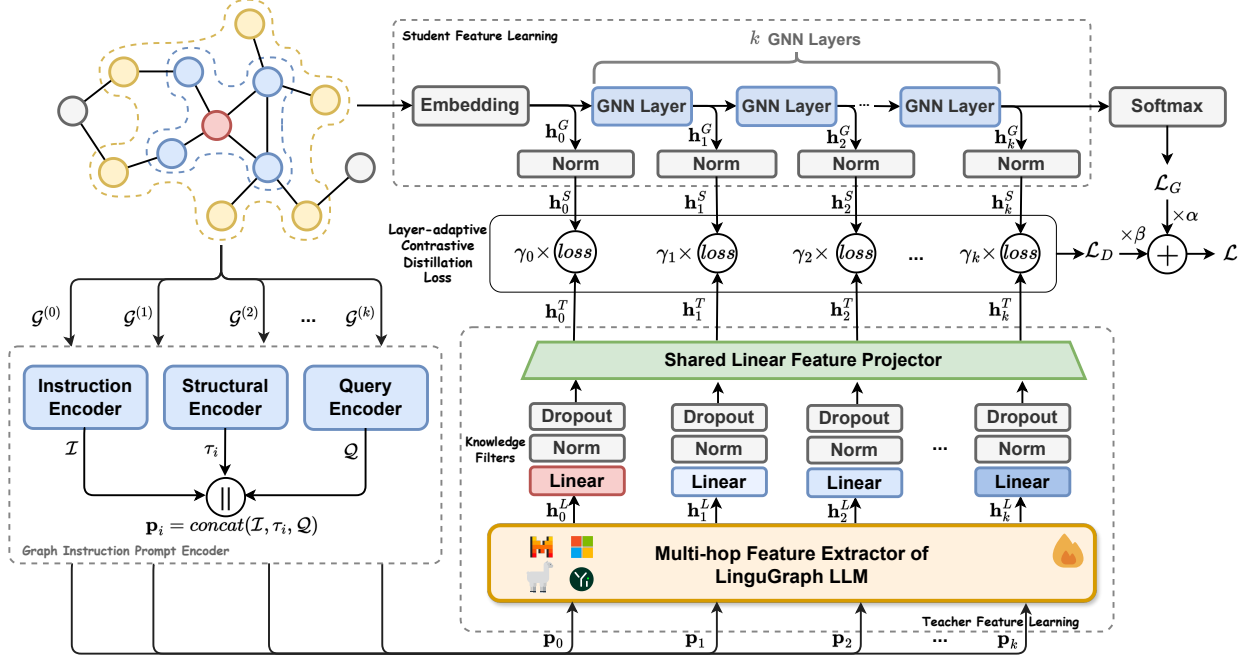


Figure 1: The proposed LinguGKD framework for TAG-oriented LLM-to-GNN knowledge distillation includes three main components: (1) Teacher feature learning using a graph-instruction-tuned LinguGraph LLM, (2) Student feature learning using a GNN, and (3) Layer-adaptive contrastive distillation to align the feature representations of both models. The lower left section depicts the process of describing the structure of a TAG with tailored graph instruction prompts for teacher feature learning.

DEFINITION 3 (LLM-TO-GNN GRAPH KNOWLEDGE DISTILLATION). An LLM-to-GNN Graph Knowledge Distillation (GKD) framework can be formulated as a quintuple: $(\mathcal{M}_T, \mathcal{M}_S, \mathcal{S}, \mathcal{F}, \mathcal{A})$, where the teacher model $\mathcal{M}_T$ is a Transformer [42]-based LLM, fine-tuned on TAG $\mathcal{G}$ for generative graph inference tasks; $\mathcal{M}_S$ denotes the student GNN specified for discriminative tasks; $\mathcal{F}$ represents the knowledge to be transferred from $\mathcal{M}_T$ to $\mathcal{M}_S$ via distillation scheme $\mathcal{S}$, learned by extraction algorithm $\mathcal{A}$. The distillation process can be formulated as follows:

$$\mathcal{F} = \{\mathcal{F}_T, \mathcal{F}_S\} = \{\mathcal{A}_T(\mathcal{M}_T, \mathcal{G}), \mathcal{A}_S(\mathcal{M}_S, \mathcal{G})\} \quad (2)$$

$$\mathcal{L}_D = \text{loss}(\mathcal{F}_S, \mathcal{F}_T) \quad (3)$$

$$\mathcal{M}_S^{\text{new}} = \mathcal{S}(\mathcal{M}_S, \mathcal{L}_D) \quad (4)$$

where $\mathcal{A}_T$ and $\mathcal{A}_S$ are the knowledge extraction algorithms of $\mathcal{M}_T$ and $\mathcal{M}_S$, respectively; $\text{loss}(\cdot)$ denotes the divergence function (e.g., Kullback-Leibler divergence), and $\mathcal{M}_S^{\text{new}}$ is the distilled student model that we ultimately require.

3 APPROACH

Figure 1 illustrates the LinguGKD framework for TAG-oriented graph knowledge distillation, highlighting three key components: teacher feature learning by the LLM, student feature learning by the GNN, and layer-adaptive contrastive distillation between the two feature sets. Before delving into these crucial modules, we elaborate on how to fine-tune a TAG-oriented Pre-trained Language Model (PLM) to understand graph structure and node semantics through our tailored instruction prompts.

3.1 TAG Instruction Tuning of Pre-trained LLM

Given a TAG $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{X})$, a center node v_i and a specific neighbor hop k , we can obtain a collection of neighbor subgraphs from structural-free up to the k -th hop: $\mathcal{G}_i^k = \{\mathcal{G}_i^{(l)}\}_{l=0}^k$, where $\mathcal{G}_i^{(k)} = (v_i, \mathcal{N}_i^{(k)}, \mathcal{E}_{\mathcal{N}_i^{(k)}}, \mathcal{X}_{\mathcal{N}_i^{(k)}})$ denotes the k -th hop subgraph, in which $\mathcal{N}_i^{(k)}$ is the set of k -hop neighbors of v_i , and $\mathcal{E}_{\mathcal{N}_i^{(k)}}$, $\mathcal{X}_{\mathcal{N}_i^{(k)}}$ are the corresponding edges and textual node attributes within the subgraph, respectively.

To enable the PLM to accurately understand graph structure and node semantics, it is essential to craft comprehensive instruction prompts for further instruction tuning. For a given $\mathcal{G}_i^{(k)}$, we define a specific instruction prompt $\mathbf{p}_k$, which consists of three components: task-specific instructions $\mathcal{I}$ that delineate the expected model action, the structural prompt τ_k that is the natural language description of the subgraph, and a task-relevant query Q , typically presented as a detailed question. The construction of $\mathbf{p}_k$ is a concatenation of these elements, as shown in the lower left part of Figure 1:

$$\mathbf{p}_k = \text{concat}(\mathcal{I}, \tau_k, Q) \quad (5)$$

Here, we employ node classification as a side task to enable the LLM to comprehend graphs. In alignment with OpenAI’s Prompt Engineering principles [33], we carefully design $\mathcal{I}$ and Q as follows:

PROMPT 1 (TAG NODE CLASSIFICATION INSTRUCTION). Implement a node classification system for $\{\{\text{type of graph}\}\}$, representing nodes as tuples (node_{id}, {degree}, {attribute}). Classify

nodes into $\{\{\text{list of classification categories}\}\}$ based on attributes and link relations. $\{\{\text{classification criteria for a specific graph}\}\}$.

PROMPT 2 (TAG NODE CLASSIFICATION QUERY). Which category should (node $\{\{id\}\}$, $\{\{node\ degree\}\}$, $\{\{node\ attributes\}\}$) be classified as?

In these templates, each node is represented by a tuple that encapsulates the node’s id, degree, and the corresponding textual attributes. The placeholders within curly braces $\{\{\}\}$ are filled based on the specific graph data. More precisely, the term $\{\{type\ of\ graph\}\}$ specifies the domain this graph belongs to, while $\{\{\text{list of classification categories}\}\}$ enumerates the potential categories for node classification. Furthermore, $\{\{\text{classification criteria for a specific graph}\}\}$ denotes the optional prior knowledge that assists the LLM in the precise generation of labels for the center node.

In crafting the structural prompt, we prioritize key elements, such as node textual attributes, node degrees, and multi-order neighbor interactions, aligned with conventional multi-layer message-passing GNNs’ principles. Referring to the strategies from [56], we develop a linguistic structural encoder f_e that transforms $\mathcal{G}_i^{(k)}$ into a detailed natural language description, represented as τ_k :

$$\tau_k = f_e(\mathcal{G}_i^{(k)}) \quad (6)$$

The prompt template of f_e is meticulously designed as follows:

PROMPT 3 (TAG STRUCTURAL PROMPT). (node $\{\{id\}\}$, $\{\{node\ degree\}\}$, $\{\{node\ attributes\}\}$) is connected within $\{\{k\}\}$ hops to $\{\{k\text{-th hop neighbors}\}\}$ through paths that may involve $\{\{intermediate paths\}\}$.

In this context, $\{\{k\text{-th hop neighbors}\}\}$ refers to the ensemble of neighbors reachable at the k -th hop in $\mathcal{N}_i^{(k)}$. Each neighbor in this list is characterized by a tuple that encapsulates its id, degree, and attributes, mirroring the representation of the central node. Furthermore, $\{\{intermediate paths\}\}$ represents the sequences of paths connecting the central node v_i to its k -th hop neighbors, encompassing the edges defined within $\mathcal{E}_{\mathcal{N}_i^{(k)}}$. This template meticulously outlines the connectivity and relational dynamics within the subgraph based on node degrees, attributes, and the specified hop distances.

Iterating the structural encoding process, we can methodically generate a structural prompt for every subgraph within $\mathcal{G}_i^k$, leading to the assembly of the k -hop structural prompt set $\mathcal{T} = \{f_e(\mathcal{G}_i^{(l)})\}_{l=0}^k$ for the central node v_i . Notably, $l = 0$ focuses exclusively on v_i ’s textual attributes. This iterative generation of structural prompts systematically captures the nuanced relational dynamics at varying degrees of connectivity, from the immediate vicinity ($l = 0$) extending to the broader k -hop network. By concatenating instruction $\mathcal{I}$ and query Q with various structural prompts, we can obtain a set of graph instruction prompts $\mathcal{P}$:

$$\mathcal{P} = \{\text{concat}(\mathcal{I}, \tau_l, Q)\}, \quad \forall \tau_l \in \mathcal{T} \quad (7)$$

Then we adopt instruction tuning [18, 39, 59] to specifically tailor PLMs for generative node classification tasks. Specifically, each prompt $\mathbf{p}_l \in \mathcal{P}$ is utilized to directly fine-tune the PLM to generate the semantic category $\mathbf{y}_l$ for the corresponding center node, without any modification of the pre-trained tokenizer and vocabulary. We

utilize negative log-likelihood loss as our objective function:

$$\mathcal{L}_T(\mathcal{P}) = - \sum_{\mathbf{p}_l \in \mathcal{P}} \sum_{j=1}^{|\mathbf{y}|} \log p(\hat{y}_j | \mathbf{p}_l, \hat{y}_{<j}) \quad (8)$$

where $\hat{y}_j$ refers to the j -th token that the LLM generated for the node label. Employing this instruction tuning methodology, we develop a highly proficient LLM adept in TAG understanding. This fine-tuned LLM, which we call LinguGraph LLM, then acts as the teacher model $\mathcal{M}_T$ in the subsequent graph knowledge distillation process.

3.2 Knowledge Distillation from LinguGraph LLM to GNN

In this section, we detail the key components of the proposed LinguGKD framework, addressing the challenge of transferring insights from complex teacher LLMs to simpler student GNNs. This process emphasizes aligning the node latent features extracted by both the LLM and the GNN within a unified latent space. The following subsections elaborate on three pivotal phases of LinguGKD’s knowledge distillation process: extracting semantically-enhanced node features via LLMs, leveraging GNNs for structural node feature extraction, and implementing layer-adaptive alignment of semantic and structural features to distill knowledge from the teacher LLM to the student GNN.

3.2.1 Teacher Feature Learning via LinguGraph LLM. We begin by leveraging the fine-tuned LinguGraph LLM $\mathcal{M}_T$ to extract semantically-enriched node features, encapsulating textual attributes and multi-order neighbor information, as depicted in the Teacher Feature Learning module in Figure 1.

Building upon insights from [51], we observe that tailored instructions significantly enhance the LLM’s proficiency in generating semantic features. Consequently, we leverage the entire instruction prompt set $\mathcal{P}$ for the extraction of node semantic features, rather than limiting it to the structural prompt set $\mathcal{T}$. Specifically, given that LLMs primarily use the transformer architecture [42], which includes an embedding layer for token embedding, an n -layer transformer with multi-head self-attention for deriving word-level nonlinear interrelations, and an output layer for specific generative tasks, we extract node semantic features as follows.

For an instruction prompt $\mathbf{p}_l \in \mathcal{P}$, we process the sequence of tokens through the embedding layer:

$$E^L = \text{Embedding}^L(\{\rho_i\}_{i=1}^{|\mathbf{p}_l|}; W_{\text{emb}}^L) \quad (9)$$

where $\text{Embedding}^L(\cdot)$ is the embedding layer of the LLM $\mathcal{M}_T$, and W_{emb}^L represents its parameters. E^L is the embedded sequence, each row of which represents a token.

Next, the embedded sequence is fed into the transformer layers:

$$H = \text{Transformer}(E^L; W_{\text{tr}}) \quad (10)$$

where $\text{Transformer}(\cdot)$ denotes the transformer module of LLM $\mathcal{M}_T$, and W_{tr} represents its parameters. The transformer layers, using multi-head self-attention, compute the contextual relationships between tokens, resulting in a contextualized feature matrix H .

Finally, we consider the feature of the last token $\rho_{|\mathbf{p}_l|}$ from the final transformer layer as the l -th order node latent feature:

$$\mathbf{h}_l^L = H_{|\mathbf{p}_l|, :} \quad (11)$$

Here, $\mathbf{h}_l^L \in \mathbb{R}^{d_L}$ encapsulates the aggregated contextual information of the entire instruction prompt, integrating neighbor details and extensive node attribute data, where d_L denotes the dimension of the node latent features extracted by the teacher LLM. Through this process, we obtain a rich semantic representation for each node, leveraging the comprehensive understanding capabilities of the LLM.

For distilling hierarchical knowledge from the teacher LLM, we then process each l -th order feature, $\mathbf{h}_l^L$, through a specialized hop-specific neural knowledge filter, $\mathcal{M}_f^L$, which is proficient in filtering pertinent layer knowledge:

$$\mathcal{M}_f^L(\mathbf{h}_l^L) = \sigma(W_l \mathbf{h}_l^L + b_l), \quad 0 \leq l \leq k \quad (12)$$

$$\hat{\mathbf{h}}_l^L = \text{LayerNorm}(\mathcal{M}_f^L(\mathbf{h}_l^L)) \quad (13)$$

where W_l, b_l are the trainable parameters for the filter $\mathcal{M}_f^L$, and σ denotes the non-linear activation function.

Subsequently, to align the node features from different hops into the same lower-dimensional distillation vector space without altering their distribution, a cross-hop shared linear feature projector, $\mathcal{M}_p$, is designed to restructure these features:

$$\mathbf{h}_l^T = \mathcal{M}_p(\hat{\mathbf{h}}_l^L; W_p, b_p) = W_p \hat{\mathbf{h}}_l^L + b_p \quad (14)$$

where $\mathbf{h}_l^T \in \mathbb{R}^{d_k}$ symbolizes the adapted l -order teacher knowledge, poised for subsequent distillation steps; W_p, b_p are the trainable parameters for $\mathcal{M}_p$.

Applying this process across all subgraph orders up to the k -th, we obtain a set of hierarchical teacher node features $\mathcal{F}_T = \{\mathbf{h}_l^T\}_{l=0}^k$, which is then utilized in the subsequent graph knowledge distillation, serving as the specific knowledge to be distilled from the teacher LLM to the student GNN.

3.2.2 Student Feature Learning via GNN. We then leverage the student GNN $\mathcal{M}_S$ to extract multi-hop node features, as shown in the Student Feature Learning module in Figure 1. The chosen student model can be any off-the-shelf GNN. Although various GNN models have different architectures [14, 24, 43, 52], they all interpret graph structures through a message-passing mechanism that enables central nodes to assimilate features from k -hop neighbors, capturing local graph structure nuances. The core principle of message-passing remains consistent: a structured process involving message construction, aggregation, and node feature updating. These stages allow GNNs to effectively represent intricate graph structures.

For a given k -hop neighbor subgraph $\mathcal{G}_i^{(k)}$ of a central node v_i , the k -order message aggregation process in GNN unfolds as follows:

$$\mathbf{h}_j^{(0)} = \text{Embedding}^G(x_j; W_{\text{emb}}^G), \quad \forall v_j \in v_i \cup \mathcal{N}_i^{(k)} \quad (15)$$

$$\mathbf{m}_{i \leftarrow j}^{(l)} = \mathcal{M}_{\text{msg}}^{(l)}(\mathbf{h}_i^{(l-1)}, \mathbf{h}_j^{(l-1)}, e_{ij}; W_{\text{msg}}^{(l)}), \quad 0 < l \leq k \quad (16)$$

$$\mathbf{h}_i^G = \mathcal{M}_{\text{update}}^{(l)}(\mathbf{h}_i^{(l-1)}, \bigoplus_{v_j \in \mathcal{N}_i} \mathbf{m}_{i \leftarrow j}^{(l)}; W_{\text{update}}^{(l)}) \quad (17)$$

where $x_j \in \mathcal{X}_{\mathcal{N}_i^{(k)}}$ represents the attribute of node v_j , $\text{Embedding}^G(\cdot)$ is the text embedding model of $\mathcal{M}_S$, such as bag-of-words or TF-IDF, converting node textual attributes into a low-dimensional vector space, establishing the initial node feature $\mathbf{h}_j^{(0)}$. The functions $\mathcal{M}_{\text{msg}}^{(l)}(\cdot)$ and $\mathcal{M}_{\text{update}}^{(l)}(\cdot)$ perform message construction and node

feature updates, respectively, during l -order message passing. $\bigoplus$ denotes a differentiable, permutation-invariant function (e.g., sum, mean, or max) to perform message aggregation. The parameters $W_{\text{msg}}^{(l)}$ and $W_{\text{update}}^{(l)}$ are the associated trainable weights. The output of the l -th message passing layer $\mathbf{h}_i^G \in \mathbb{R}^{d_G}$ represents the l -th order feature of the central node v_i , reflecting its l -order neighbor structure and attributes.

Subsequently, we synchronize the node features into the unified distillation vector space via a normalization layer:

$$\mathbf{h}_l^S = \text{Norm}(\mathbf{h}_l^G), \quad 0 \leq l \leq k \quad (18)$$

where $\mathbf{h}_l^S \in \mathbb{R}^{d_k}$ denotes the student knowledge, and $\text{Norm}(\cdot)$ is a normalization function, typically batch or layer normalization. Applying this process across all subgraph orders up to the k -th, we obtain a set of hierarchical student node features $\mathcal{F}_S = \{\mathbf{h}_l^S\}_{l=0}^k$.

In the next phase, we focus on aligning the student GNN's learned node features $\mathcal{F}_S$ with those from the teacher LLM, $\mathcal{F}_T$. This alignment ensures that the LLM's rich semantic knowledge is effectively integrated into the GNN, enhancing its capability to process graph-structured data.

3.2.3 Layer-Adaptive Contrastive Distillation. In graph inference, the relevance of node features varies significantly with the order of their neighbors: structural-free features highlight a node's core attributes, lower-order features elucidate direct interactions, and higher-order features provide insight into distant connections, which are essential for understanding a node's role within the graph. In different downstream tasks, the importance of features encoding information from various neighbor orders can differ significantly. For instance, tasks such as community detection may rely more on higher-order features [20], while node classification might benefit more from lower-order features [24].

Given these variations, a one-size-fits-all distillation approach may fail to capture the nuanced importance of features at different hops. Therefore, to fully leverage the teacher LLM's comprehensive understanding of different neighbor orders, we propose a Layer-Adaptive Contrastive Distillation mechanism within the LinguGKD framework. This approach tailors the distillation process to the importance of node features at each hop, ensuring the student GNN effectively captures the teacher LLM's nuanced knowledge of both local and global graph structures. This layer-adaptive strategy is essential for optimizing the student GNN's performance across diverse tasks by facilitating precise and task-relevant knowledge transfer.

To achieve effective knowledge transfer, it is crucial to align the feature spaces of the teacher LLM and the student GNN. Contrastive learning with infoNCE loss [32] is particularly well-suited for this task because it encourages the alignment of similar (positive) feature pairs while ensuring that dissimilar (negative) pairs are distinguishable. By leveraging contrastive learning, we can effectively measure and minimize the divergence between the layer-wise features extracted by both models, thereby ensuring that the student GNN can accurately mimic the teacher LLM's deep semantic understanding.

To design an effective contrastive distillation loss, we consider the following steps: For each node v_i and its corresponding l -th order feature, the positive sample is the same node's feature as learned by both the teacher LLM and the student GNN. This ensures that the features of the same node are aligned across different models.

The negative samples are features of nodes from different categories compared to the center node v_i . This selection helps in maintaining a clear distinction between different classes, thereby improving the classification accuracy of the student GNN. Formally, given a node v_i , its l -th order feature from the teacher LLM, $\mathbf{h}_l^T$, and from the student GNN, $\mathbf{h}_l^S$, the positive pair is defined as:

$$(\mathbf{h}_l^S, \mathbf{h}_l^T) \quad (19)$$

For each positive pair, we sample N negative pairs $\{(\mathbf{h}_l^S, \mathbf{h}_l^{T*})\}$, where $\mathbf{h}_l^{T*}$ is the l -th order feature of a node from a different category. The infoNCE loss for each l -th order feature, controlled by a temperature parameter t , is expressed as follows:

$$\mathcal{L}_D^l = -\mathbb{E} \left[\log \frac{\exp(\text{sim}(\mathbf{h}_l^S, \mathbf{h}_l^T)/t)}{\sum_{m=1}^N \exp(\text{sim}(\mathbf{h}_l^S, \mathbf{h}_l^{T*})/t)} \right] \quad (20)$$

where $\text{sim}(\cdot, \cdot)$ denotes a similarity function, such as cosine similarity. The temperature parameter t controls the smoothness of the probability distribution, ensuring that the model focuses on hard negative samples that are more challenging to distinguish from the positives.

Recognizing the unique importance of different-order neighbor structures in various downstream tasks, we then introduce a trainable distillation factor γ_l for each layer's distillation loss, which allows the model to adaptively focus more on the layers that are more critical for the specific task at hand. The overall layer-adaptive contrastive distillation loss is then computed as the weighted sum of the layer-specific contrastive losses:

$$\mathcal{L}_D = \sum_{l=0}^k \gamma_l \mathcal{L}_D^l \quad (21)$$

Here, each order's distillation factor γ_l ensures a balanced knowledge distillation, enabling the effective transfer of complex semantic and structural insights from the LLM to the GNN.

3.3 Model Training

In training the student GNN, acknowledging the different inference frameworks of the teacher and student models, it is essential to not only distill knowledge from the teacher LLM but also to train a task-specific prediction layer for the GNN. Therefore, we approach the student GNN's training as a multi-task joint optimization challenge. For instance, in the node classification scenario, we use a fully connected layer as the classifier:

$$\hat{y} = \text{softmax}(W_G \mathbf{h}_k^S + b_G) \quad (22)$$

where $\mathbf{h}_k^S$ denotes the output of the k -th layer of the GNN, $\hat{y}$ represents the predicted node label, and W_G and b_G are the weights and biases of the fully connected layer.

Subsequently, the node classification loss function, formulated as cross-entropy, is computed as:

$$\mathcal{L}_G = - \sum_{i=1}^{|\mathcal{D}_{tr}|} y_i \log(\hat{y}_i) \quad (23)$$

where y_i denotes the actual node label, and $\hat{y}_i$ is the predicted probability for each category. $\mathcal{D}_{tr}$ is the training set.

Table 1: Dataset Statistics

	Cora	PubMed	Arxiv
# Node	2,708	19,717	169,343
# Edge	5,429	44,338	1,166,243
# Class	7	3	40
# Features	1433	500	128
Embedding Tech.	BoW	TF-IDF	Skip-gram
$ \mathcal{D}_{tr} : \mathcal{D}_{val} : \mathcal{D}_{test} $	6:2:2	6:2:2	5.4:1.8:2.8

The overall training objective integrates the KD loss $\mathcal{L}_D$ with the classification loss $\mathcal{L}_G$, obtaining a joint loss function:

$$\mathcal{L} = \alpha \mathcal{L}_G + \beta \mathcal{L}_D \quad (24)$$

where α and β are tunable factors for adaptively balancing the influence of knowledge distillation loss and downstream task loss on the training process.

Finally, the student GNN undergoes end-to-end training using a mini-batch AdamW optimizer, which is dedicated to optimizing model parameters efficiently for robust performance.

4 EXPERIMENTS

4.1 Datasets and Backbone Models

Datasets. We validated the effectiveness of our proposed LinguGKD framework through node classification experiments on three widely-adopted benchmark datasets: Cora, PubMed [55], and Arxiv [19]. These datasets represent academic papers as nodes and citations as edges. The node attributes consist of titles and abstracts, encapsulating the core content of each paper. For knowledge distillation and GNN training, we utilized the default node embeddings generated by various techniques (e.g., bag of words (BoW), TF-IDF) inherent to these datasets without any alterations.

Due to the lack of initial text attributes for each node in the original datasets, we reconstructed titles, abstracts, and other text attributes for each node following the method described in [16] for graph instruction tuning of the teacher LLM.

For dataset partitioning, we followed the split strategy adopted in [16, 56]. Specifically, we applied a 6:2:2 split for the Cora and PubMed datasets, while for the Arxiv dataset, we used the 5.4:1.8:2.8 split as in the OGB open benchmark [19]. Comprehensive dataset statistics are summarized in Table 1.

Backbone Models. To verify the effectiveness and generality of our proposed LinguGKD framework, we selected multiple LLMs with different architectures as teachers and GNNs as students. Specifically, we chose Mistral-7B [22], Llama2-7B [41], and Llama3-8B [2] as the teacher models. For student GNNs, we selected GCN [14], GAT [43], GraphSAGE [14], and GIN [52], known for their effectiveness and efficiency in graph-based tasks.

4.2 Experimental Settings

Instruction Tuning of PLM. During the LLM's fine-tuning phase, for each center node v_i , we defined a maximum neighboring sub-graph order k of 3, constructing instruction prompts $\mathcal{P} = \{\mathcal{P}_l\}_{l=0}^3$ to cover structural prompts from structure-free to 3rd-hop subgraphs.

Table 2: Overview of Experimental Settings

Parameter	Value
Instruction Tuning of PLM	
Maximum neighboring subgraph order (k)	3
Maximum number of instruction prompts per hop (θ)	2
Maximum sequence length (s_{max})	512
Hidden feature dimension (d_L)	4096
Batch size (mb_L)	2
Gradient accumulation (g_{acc})	2
Learning rate (lr_L)	2×10^{-4}
Training duration (ep_L)	1 epoch
LoRA parameters	$lora_r=64, lora_\alpha=32$
Graph Knowledge Distillation	
Learning rate (lr_G)	1×10^{-4}
Batch size (mb_G)	32
Training epochs (ep_G)	500
Feature dimension (d_G, d_k)	[64, 128, 256, 512, 1024]

We standardized the maximum number of instruction prompts per hop (θ) at 2, resulting in totals of 62,216, 121,580, and 2,000,123 instruction prompts for Cora, PubMed, and Arxiv, respectively.

We utilized pre-trained models from *huggingface.co*. The LLM input’s maximum sequence length (s_{max}) was capped at 512, with a hidden feature dimension (d_L) of 4096. Training settings included a minimal batch size (mb_L) of 2, gradient accumulation (g_{acc}) also set at 2, and a learning rate (lr_L) of 2×10^{-4} , all within a concise training duration of 1 epoch (ep_L). We integrated LoRA and 4-bit quantization techniques, with LoRA’s parameters set to $lora_r=64$, dropout at 0.1, and alpha ($lora_\alpha$) to 32. This adjustment resulted in a total of approximately 1,800,000 trainable parameters, including elements like q_proj , k_proj , v_proj , o_proj , and lm_head .

In the node label generation steps of LinguGraph LLM, each node underwent classification for every instructional prompt via a greedy search algorithm. A majority voting scheme was then applied, where the most frequently appearing classification across prompts was selected as the final prediction, ensuring a balanced and democratically derived outcome.

Graph Knowledge Distillation. For distilling graph knowledge from LinguGraph LLM to GNNs, across all benchmark datasets, we standardized the learning rate (lr_G) at 1×10^{-4} , batch size (mb_G) at 32, and extended the training to 500 epochs (ep_G). The performance of our LinguGKD was assessed across different message-passing layers [0, 1, 2, 3].

For simplicity, we aligned the feature dimension of the GNN (d_G) with that of the distilled knowledge (d_k). We then explored the effects of varying the hidden feature dimensions of GNNs and distilled knowledge at [64, 128, 256, 512, 1024].

For each central node v_i within a specified neighbor order l , our experimental setup involved generating θ unique instruction prompts. This led to the extraction of θ l -hop features $\{\mathbf{h}_{l,(1)}^L, \dots, \mathbf{h}_{l,(\theta)}^L\}$ from the LinguGraph LLM for v_i . During the layer-adaptive knowledge

distillation phase, we utilized an average pooling operation to consolidate these features into a single representation $\mathbf{h}_l^L$ that accurately reflects the l -th order neighborhood’s characteristics. The overall experimental settings are summarized in Tabel 2.

4.3 Experimental Results and Analyses

4.3.1 Comparison Performance Analyses. To validate the effectiveness of our proposed LinguGKD, we report the accuracy and F1 score of node classification over different datasets to evaluate the performance of various graph learning models. We selected a series of representative single-model graph learning approaches as baselines from three corresponding leaderboards^{2 3 4}, ranging from simple to advanced architecture designs. These include message-passing-based GNNs (such as BernNet [15], RevGAT [25], etc.) and more complex graph Transformer models (such as Graphormer [57], E2EG [9], etc.). Table 3 shows the node classification results.

Effectiveness Analysis of LinguGraph LLMs Compared to Baselines. The experimental results demonstrate that, after graph instruction tuning, the LinguGraph LLMs exhibit strong capabilities in semantic and entity relationship understanding and generalizes well across different datasets. Compared to baseline GNNs, the tuned LinguGraph LLMs achieve state-of-the-art results on all datasets (e.g., LinguGraph-Llama3 (8B) achieved 91.51%, 95.59%, and 79.73% accuracy on the Cora, PubMed, and Arxiv datasets, respectively). Additionally, the performance of the LinguGraph LLM improves with the increase in pre-trained LLM corpus and model parameters. For instance, Llama3-8B, with more model parameters and a larger pre-training corpus than Llama2-7B and Mistral-7B, consistently performs well across multiple datasets. These results strongly support the assertion in [56] that LLMs have the potential to become the next generation foundational model for future graph learning.

In graph knowledge distillation frameworks, student models often experience performance degradation compared to teacher models [4, 23, 37], especially without additional training data. Therefore, selecting an exceptional teacher model is crucial for enhancing the performance of student GNNs. The experimental results above motivate our choice of LLM as the teacher model for graph knowledge distillation, ensuring more effective student GNNs.

Performance Gains Analysis of Distilled GNNs Over Baselines.

From the results, we can observe that the proposed LinguGKD framework can be seamlessly applied to different combinations of teacher LLMs and student GNNs, significantly improving the performance of various student GNNs by extracting knowledge from LLMs without requiring additional training data or modifications to the GNN architecture. The average distillation gains of various student GNNs range from 2.79% in PubMed to 4.61% in Cora. For instance, on the Cora dataset, the accuracy of the GCN model distilled with LinguGraph-Llama3 (8B) increased to 90.77%, compared to 86.53% for the vanilla GCN model.

Moreover, compared to other advanced GNNs and graph Transformer models, the LinguGKD framework enables basic GNN models to achieve competitive performance with these more complex

²<https://paperswithcode.com/sota/node-classification-on-cora-60-20-20-random>

³<https://paperswithcode.com/sota/node-classification-on-pubmed-60-20-20-random>

⁴https://ogb.stanford.edu/docs/leader_nodeprop/

Table 3: Node classification performance of various graph learning models across selected datasets, with the highest-performing outcomes in *bold*, second-best scores highlighted in gray, and best baseline performances underlined. The prefix *LinguGraph-* represents teacher LLMs obtained by fine-tuning different PLMs with graph instruction prompts, while GNNs with different subscripts represent student models distilled from the corresponding teacher LLMs indicated by the subscripts. The term *Avg. Dist. Gains*. refers to the average knowledge distillation gains obtained by different student GNNs.

Methods	Cora		PubMed		Methods	Arxiv	
	Acc.↑	F1↑	Acc.↑	F1↑		Acc.↑	F1↑
GCN [24]	86.53±0.92	85.66±0.78	86.12±0.93	85.64±0.82	GCN [24]	71.74±0.21	71.04±0.37
GAT [43]	86.12±0.95	85.05±0.88	85.49±0.76	84.89±0.71	GAT [43]	73.66±0.33	72.44±0.19
GraphSAGE [14]	87.08±0.85	85.96±0.73	87.69±0.92	87.38±0.68	GraphSAGE [14]	71.19±0.26	70.87±0.45
GIN [52]	86.60±0.91	85.37±0.74	85.84±0.92	85.31±0.63	GIN [52]	71.62±0.47	71.13±0.33
SGC-v2 [47]	85.48±1.48	85.04±0.69	85.36±0.52	84.96±0.77	DeeperGCN [26]	71.92±0.16	71.24±0.39
BernNet [15]	88.52±0.95	87.96±0.85	88.48±0.41	87.52±0.79	GTAN [48]	72.97±0.17	71.77±0.22
FAGCN [3]	88.85±1.36	87.92±0.65	89.98±0.54	88.72±0.53	UniMP [38]	73.11±0.20	72.14±0.38
GCNII [5]	88.93±1.37	87.58±0.71	89.80±0.30	88.96±0.62	GCNII [5]	72.74±0.00	72.22±0.44
RevGAT [25]	89.11±0.00	87.65±0.58	88.50±0.05	87.12±0.73	RevGAT [25]	<u>74.02±0.18</u>	<u>73.56±0.29</u>
ACM-GCN+ [31]	<u>89.75±1.16</u>	<u>88.94±0.54</u>	<u>90.96±0.62</u>	<u>89.77±0.51</u>	E2EG [9]	73.62±0.14	72.96±0.26
GraphTransformer [11]	86.42±0.82	85.96±0.67	88.75±0.16	87.91±0.59	SGFormer [49]	72.63±0.13	71.58±0.42
Graphormer [57]	80.41±0.30	79.98±0.56	88.24±1.50	87.52±0.71	Graphormer [57]	72.81±0.23	72.14±0.39
LinguGraph-Mistral (7B)	87.82±0.88	87.47±0.72	93.71±0.69	93.37±0.52	LinguGraph-Mistral (7B)	76.07±0.53	76.02±0.44
LinguGraph-Llama2 (7B)	88.19±0.83	88.12±0.73	94.09±0.78	93.55±0.61	LinguGraph-Llama2 (7B)	75.67±0.52	75.60±0.41
LinguGraph-Llama3 (8B)	91.51±0.46	91.53±0.18	95.59±0.29	95.55±0.10	LinguGraph-Llama3 (8B)	79.73±0.18	79.29±0.56
GCN _(Mistral)	90.04±0.64	89.62±0.58	88.92±0.71	88.47±0.69	GCN _(Mistral)	73.55±0.49	73.27±0.35
GCN _(Llama2)	90.59±0.71	89.62±0.66	88.97±0.82	88.56±0.71	GCN _(Llama2)	73.87±0.22	73.87±0.61
GCN _(Llama3)	90.77±0.28	90.35±0.37	89.76±0.44	89.46±0.37	GCN _(Llama3)	74.68±0.45	74.29±0.32
GAT _(Mistral)	89.85±0.62	89.19±0.52	88.08±0.62	87.53±0.47	GAT _(Mistral)	74.72±0.47	74.55±0.42
GAT _(Llama2)	90.33±0.67	89.72±0.59	87.93±0.28	87.42±0.36	GAT _(Llama2)	74.92±0.14	74.48±0.28
GAT _(Llama3)	91.51±0.35	91.45±0.58	88.31±0.76	87.93±0.65	GAT _(Llama3)	75.71±0.41	75.06±0.36
GraphSAGE _(Mistral)	90.59±0.82	89.85±0.75	90.11±0.69	89.77±0.54	GraphSAGE _(Mistral)	72.85±0.42	72.87±0.41
GraphSAGE _(Llama2)	90.22±0.77	89.89±0.19	89.96±0.50	89.67±0.34	GraphSAGE _(Llama2)	72.53±0.61	72.42±0.49
GraphSAGE _(Llama3)	91.70±0.51	91.08±0.62	90.14±0.56	89.96±0.48	GraphSAGE _(Llama3)	75.38±0.38	75.22±0.32
GIN _(Mistral)	89.67±0.71	88.64±0.54	87.83±0.62	87.27±0.56	GIN _(Mistral)	73.40±0.42	73.63±0.34
GIN _(Llama2)	90.26±0.67	89.20±0.48	87.73±0.29	87.20±0.30	GIN _(Llama2)	73.71±0.25	73.42±0.10
GIN _(Llama3)	91.33±0.28	91.05±0.53	89.22±0.79	88.87±0.61	GIN _(Llama3)	75.64±0.46	75.28±0.39
Avg. Dist. Gains	4.61%	5.22%	2.79%	2.84%	Avg. Dist. Gains	3.85%	4.22%

models through knowledge distillation, and even outperform them in certain scenarios. For instance, the GAT model distilled by LinguGraph-Llama3 (8B) on the Cora dataset achieved 91.51% accuracy, surpassing the performance of more complex models such as RevGAT (89.11%) and GraphTransformer (86.42%). This demonstrates that the LinguGKD framework can significantly enhance the performance of basic GNNs without increasing model complexity, making them more efficient and practical for real-world applications.

This notable performance improvement indicates that the LinguGKD framework can effectively transfer the deep semantic knowledge and complex graph structural understanding from the teacher LLM to the student GNN, thus improving its performance in node classification tasks. Furthermore, student GNNs distilled from higher-performing teacher LLMs also achieve higher accuracy and F1 scores in their respective tasks, which indicates significant potential for leveraging the rapid advancements in pre-trained LLMs to

continuously enhance the performance of student GNNs, thereby improving the efficiency of applications in production environments.

Analysis of Knowledge Distillation Effectiveness Across Different Datasets. The effectiveness of knowledge distillation varies across different datasets. For instance, on the Cora dataset, the improvement in student GNNs was substantial, with some models even surpassing the performance of the teacher LLM, such as GraphSAGE_{Llama3} achieving an accuracy of 91.70%, surpassing its teacher LinguGraph-Llama3 (8B) which had 91.51%. In contrast, the improvement on the PubMed dataset was relatively modest. The potential reason is that the LLM’s predictions for node classification are primarily derived from the node’s textual attributes, meaning it performs better in a structural-free context, with graph structural information contributing less significantly. The training of student GNNs used pre-extracted node semantic embeddings provided by the datasets, which lost significant information compared to the raw

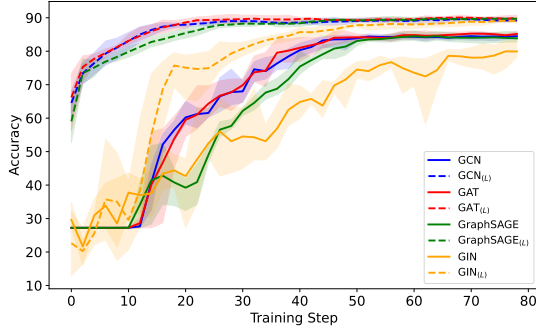


Figure 2: Convergence efficiency of vanilla GNNs and distilled GNNs.

text inputs used by the LLM, resulting in GNNs relying more on graph structural information for classification.

Specifically, in the Cora dataset, the overlapping relevance between class labels in the paper titles and abstracts results in nodes that can reasonably belong to multiple categories, making it easier for the teacher LLMs to confuse these categories. The distilled GNNs, on the other hand, not only retained the advantage of understanding graph structure but also gained the semantic understanding capability of the teacher LLM through the layer-adaptive feature alignment, leading to noticeable performance improvements. In contrast, the PubMed dataset has fewer categories, which are well-reflected in the node’s textual attributes, allowing the LLM to achieve excellent performance. However, the student GNNs, constrained by the small initial feature dimensions provided by the dataset and significant loss of semantic information, faced limitations in aligning their hierarchical features with those of the LLMs due to extensive cross-category citations, resulting in less pronounced improvements post-knowledge distillation.

In summary, our proposed LinguGKD framework excels in knowledge distillation, significantly enhancing the performance of student GNNs. The teacher LLM, after graph instruction tuning, demonstrates strong semantic and entity relationship understanding capabilities and generalizes well to different graph datasets, providing robust support for student GNNs. The experimental results also indicate that basic GNN models can achieve performance comparable to, or even surpass, complex graph learning models after distillation through LinguGKD. With the continuous development and performance improvement of pre-trained LLMs, the LinguGKD framework is poised to further enhance the performance of GNNs, thereby advancing the efficiency of applications in production environments.

4.3.2 Convergence Efficiency of Vanilla GNNs vs. Distilled GNNs. Figure 2 illustrates that GNNs optimized through knowledge distillation not only achieve higher classification accuracy but also exhibit faster convergence rates. Here, we select LinguGraph-Llama2 as the teacher. The distilled GNNs, distinguished by the subscript (_L), quickly reach high accuracy early in the training process, significantly outperforming their undistilled counterparts.

This enhanced convergence is primarily attributed to our joint optimization of knowledge distillation and downstream tasks in the

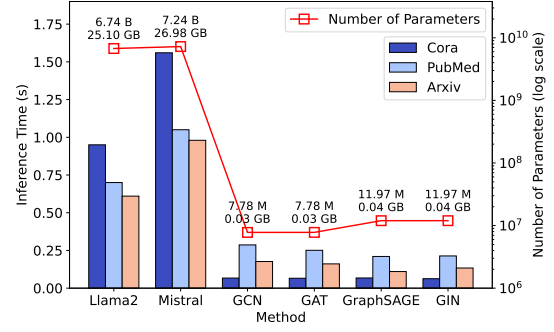


Figure 3: The number of model parameters and inference times.

training procedure, in which GNNs are trained to fit the node feature distributions learned by the teacher LLMs, facilitating rapid convergence. Teacher LLMs deliver high-quality node feature representations due to their robust semantic understanding and contextual modeling capabilities. These representations encapsulate complex semantic relationships and extensive contextual information among nodes. When student GNNs learn these feature distributions during the distillation process via the proposed LinguGKD framework, they can significantly reduce the training iterations required to achieve stable high accuracy, resulting in accelerated convergence.

In summary, knowledge distillation not only enhances the classification performance of GNNs but also significantly accelerates model convergence. This advantage renders distilled GNNs more efficient for practical applications, enabling them to complete training in a shorter time while maintaining excellent performance.

4.3.3 Application Trade-off of LinguGraph LLM vs. Distilled GNN. Figure 3 shows the differences between teacher LLMs and student GNNs in terms of model parameters, storage requirements, and inference latency. The right y-axis delineates the parameter quantity and storage requirements of various models through a line graph, while the left y-axis showcases the inference latency for these models across different datasets via a bar graph.

From the figure, we can observe that teacher LLMs have substantially higher parameter counts and storage needs compared to student GNNs. Specifically, Llama2 and Mistral have parameter counts of 6.74B and 7.24B, respectively, while student GNNs have only a few million parameters. In terms of storage, teacher LLMs require over 25GB, whereas student GNNs require just 0.03GB to 0.04GB. Additionally, the inference time for teacher LLMs exceeds 0.5 second on the Cora, PubMed, and Arxiv datasets, whereas the inference time for student GNNs is much smaller.

In practical applications, the choice between using an LLM and a distilled GNN involves several trade-offs. If the application demands the highest possible performance and accuracy, and there are ample computational and storage resources available, deploying a teacher LLM might be preferable. Furthermore, if the application involves tasks that combine natural language processing and graph-based tasks, using a teacher LLM may be essential due to its advanced capabilities in both natural language and graph understanding. However, if the application requires real-time performance or operates under limited resources, a distilled GNN is a more suitable

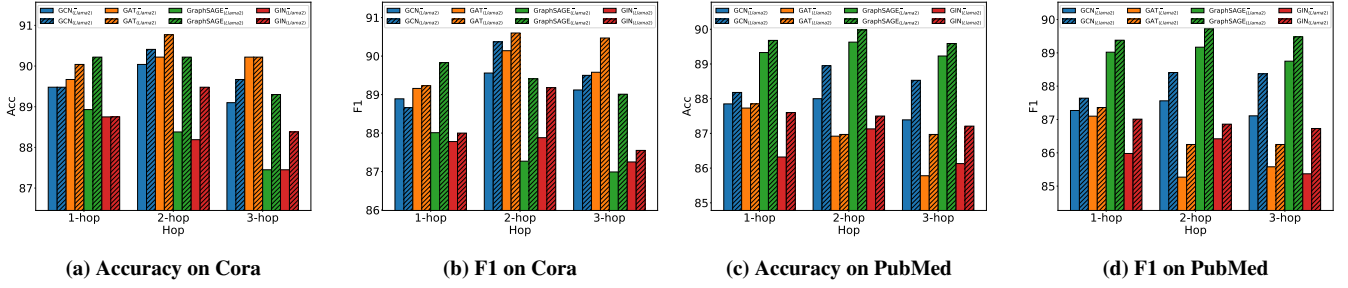


Figure 4: Node classification performance comparison of various GNN models distilled with and without layer-adaptive graph knowledge distillation on Cora & PubMed.

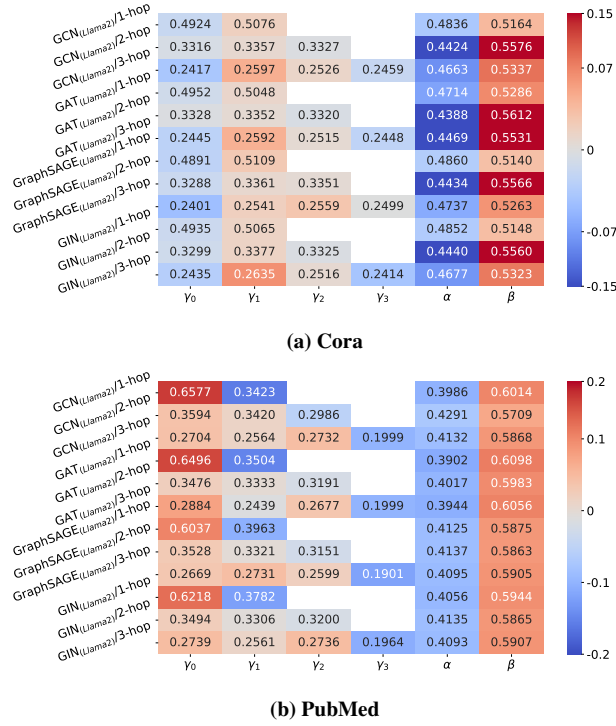


Figure 5: A heatmap visualization of the layer-adaptive factors γ_l and classification-distillation loss weights α and β during the training of distilled GNN models on Cora and PubMed datasets. The color depth indicates the differences in various factors and the average value.

choice. Distilled GNNs not only offer advantages in inference speed and resource consumption but also inherit some capabilities of the teacher LLM through knowledge distillation, striking a good balance between performance and efficiency.

4.4 Ablation Study

Necessity Analysis for Graph Instruction Tuning of PLM. Table 4 illustrates the accuracy of node classification on different datasets for various LinguGraph LLMs and their pre-trained versions. The

Table 4: Accuracy of node classification on different datasets for various pre-trained LLMs and their versions after graph instruction tuning.

Methods	Cora	PubMed	Arxiv
Mistral (7B)	8.76	24.91	10.28
LinguGraph-Mistral (7B)	87.82	93.71	76.07
Llama2 (7B)	9.23	25.04	10.72
LinguGraph-Llama2 (7B)	88.19	94.09	75.67
Llama3 (8B)	14.92	30.17	15.31
LinguGraph-Llama3 (8B)	91.51	95.59	79.73

results indicate a substantial improvement in performance across all datasets following the tuning process.

Pre-tuned models exhibited low accuracy scores: around 8-15% on the Cora dataset, 25-30% on the PubMed dataset, and 10-15% on the Arxiv dataset. Since the pre-training of LLMs typically does not include corpora related to graph understanding, they only have a basic understanding of graphs and are prone to issues such as repetition [13]. However, after graph instruction tuning, their accuracies increased dramatically, with LinguGraph-Mistral (7B) averaging around 86%, LinguGraph-Llama2 (7B) around 86%, and LinguGraph-Llama3 (8B) reaching up to 89% across these datasets.

These results underscore the effectiveness of graph instruction tuning with specifically designed graph instruction prompts in improving the LLMs’ ability to understand and classify nodes within graph structures, validating the efficacy of this approach in constructing powerful teacher LLMs for knowledge distillation.

Effectiveness Analysis of Layer-Adaptive Contrastive Distillation.

To validate the effectiveness of the proposed layer-adaptive contrastive distillation, we conducted a set of experiments comparing the impact of using versus not using the layer-adaptive distillation strategy on our proposed LinguGKD framework. We constructed a variant of LinguGKD, named LinguGKD⁻, which only aligns the last-order node features extracted by the LinguGraph LLM with the features output by the last message-passing layer of the GNN in the distillation space. We used LinguGraph-Llama2 (7B) as the teacher model and conducted comparative experiments on the Cora and PubMed datasets. The node classification performance of GNNs

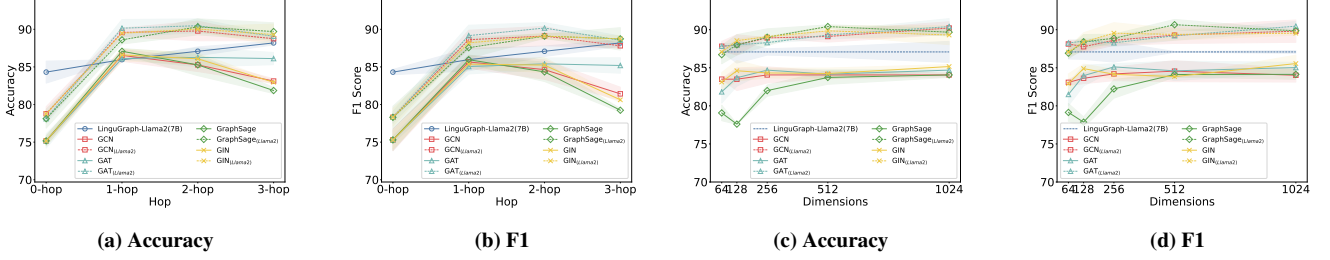


Figure 6: The results of performance impacts as the variations of neighbor orders and hidden feature dimensions.

distilled using the LinguGKD framework and the LinguGKD⁻ variant across different hops are shown in Figure 4, in which GNNs with the ⁻ superscript indicate they were distilled from the LinguGKD⁻.

From the results, we observe that GNNs distilled with the full LinguGKD framework consistently achieve higher performance on both the Cora and PubMed datasets compared to those distilled with the LinguGKD⁻ variant. This performance improvement indicates that the layer-adaptive contrastive knowledge distillation strategy effectively enhances the model’s ability to leverage multi-hop information and capture nuanced features necessary for accurate node classification. Theoretically, this approach allows the student GNN to progressively align with the hierarchical representations captured by the teacher LLM at different layers, facilitating the transfer of complex, multi-layered knowledge. This alignment helps the student GNN better mimic the teacher LLM’s deep semantic understanding and contextual modeling capabilities, resulting in more robust and accurate node representations. Consequently, student GNNs distilled with the layer-adaptive strategy demonstrate superior performance, validating the robustness and generalizability of our approach.

Impact of Layer-Adaptive Distillation and Loss Weights on Model Convergence. To further elucidate the contributions of layer-adaptive contrastive knowledge distillation and the interplay between distillation loss and downstream task loss to model convergence during the joint optimization process, we present the layer-adaptive factors γ_l and classification-distillation loss weights α and β on the Cora and PubMed datasets in Figure 5. The heatmaps show that the distillation loss weights (β) are consistently high across both datasets, indicating the significant role of distillation in accelerating GNN convergence. This trend underscores the importance of contrastive distillation in the training process, as GNNs distilled with higher β values converge faster and perform better.

The distribution of layer-adaptive factors γ_l varies across different datasets. In Cora, there is a distinct emphasis on first-order neighbor knowledge during distillation, reflected by higher γ_1 values. This suggests that for Cora, immediate neighborhood information is crucial, likely because the textual attributes and node labels have lower semantic relevance, making structural information vital for effective classification. Conversely, the PubMed dataset emphasizes structure-free features. This suggests that semantic features are more significant in PubMed, where the higher semantic relevance between node textual attributes and labels makes the LLM’s semantic understanding crucial. This analysis highlights the critical role of the

proposed layer-adaptive contrastive distillation strategy in leveraging the unique structural and semantic characteristics of each dataset.

4.5 Impact of Hyperparameters on Performance

Based on the Cora dataset, we investigated how varying hyperparameters, specifically neighbor orders (k) and hidden feature dimensions (d_G) of GNNs, affects model performance in node classification within TAG. The hidden feature dimensions of the teacher LLMs were kept constant at 4096, as per the original model design.

Figures 6a and 6b reveal that teacher LLMs consistently outperformed vanilla GNNs, regardless of neighbor hops, highlighting their superior semantic processing ability. Notably, even with no neighbor information (0-hop), LLMs showed a significant edge over GNNs. Under the LinguGKD framework, distilled GNNs surpassed original GNNs in all neighbor hops, demonstrating the effective transfer of multi-hop knowledge from LLMs to GNNs. However, while LLMs benefitted from increasing neighbor orders, GNNs experienced performance declines past the 2-hop mark due to over-smoothing. Distilled GNNs alleviated this issue but faced longer fine-tuning times for teacher LLMs with increased order, leading us to choose a 2-hop setting for a balance of efficiency and effectiveness.

Figures 6c and 6d provide insights into the impact of hidden feature dimensions on model performance. Vanilla GNNs improved up to a 128-dimension limit, beyond which their performance plateaued. In contrast, distilled GNNs continued to show enhanced performance with higher dimensions, owing to their improved capacity for semantic and structural understanding from LLMs. Consequently, we set the hidden feature dimension at 1024 in our experiments to maximize the benefits from the teacher models.

5 RELATED WORK

5.1 LLMs based Graph Learning

Recent advancements in graph learning have been significantly enriched by the integration of LLMs, marking a notable evolution in the field. Research in this domain can be primarily categorized into two distinct approaches: LLM as Enhancer (LaE) [6, 16, 45] and LLM as Predictor (LaP) [12, 44, 56], distinguished by their degree of integration with graph-structured data.

The LaE approach enhances node embedding quality in GNNs by leveraging the semantic processing capabilities of LLMs, addressing traditional GNNs’ limitations in extracting semantic features from TAGs. For instance, TAPE [16] generates interpretive explanations and pseudo-labels, enriching the graph’s textual attributes

and fine-tuning a smaller-scale language model to transform textual semantics into robust node embeddings. Similarly, Chen et al. [6] propose the Knowledge Entity Augmentation (KEA) strategy, employing LLMs to generate knowledge entities with textual descriptions, enhancing graph nodes with nuanced semantic information. Other notable methods, such as Qian et al. [36], produce semantically-rich interpretations of strings for fine-tuning a compact language model, showing potential in fields like pharmaceutical discovery. Wei et al. [45] enhance user-item interaction edges in recommendation systems, creating a richer edge dataset and improving system precision. Several studies have also explored the direct application of LLMs in producing text-based node embeddings for GNNs. The GIANT method [7] refines language models through a self-supervised learning framework, utilizing XR-Transformers [58] to address multi-label classification challenges in link prediction. Similarly, Duan et al. [10] and Zhu et al. [61] enhance PLMs using link prediction analogues to improve structural awareness. Huang et al. [21] integrate a graph-tailored adapter at the terminus of PLMs to extract graph-aware node features, generating interpretable node representations. Tan et al. [40] introduce an unsupervised technique for universal graph representation learning, converting random walks on graphs into text sequences for fine-tuning LLMs.

The LaP methodologies utilize LLMs directly for prediction tasks in graph contexts, including classification and inference. Recent research leverages LLMs pre-trained on large-scale corpora for encoding graph structures in natural language, enabling direct inference. Studies by Wang et al. [44] and Fatemi et al. [12] explore LLMs’ ability to process textual descriptions of graphs, highlighting their potential and limitations. Ye et al. [56] propose scalable prompting techniques, creating direct relational links between nodes through natural language, outperforming traditional GNNs in node classification tasks across benchmarks.

Collectively, these advancements underscore the potential of LLMs in graph learning, skillfully deciphering both semantic content and complex graph structures, and introducing innovative methodologies to the field.

5.2 Graph Knowledge Distillation

In the realm of graph knowledge distillation, the strategic transfer of intricate knowledge from complex teacher models to simpler student models is crucial for enhancing GNNs’ effectiveness and efficiency. This technique maintains the student model’s lightweight nature while striving to emulate the teacher model’s advanced behavior. Predominantly, research in this field is categorized into three areas based on the type of knowledge distilled: output logits [1, 17, 46], latent features [4, 23, 37], and graph structure [8, 50, 53].

Research focusing on output logits in graph knowledge distillation aims at transferring the final output representations from the teacher model to the student model. He et al. [17] proposed the Scalable and Effective Knowledge Distillation Framework (SGKD) for graph representation learning, which includes feature propagation to provide MLPs with graph structure-aware features. Ahluwalia et al. [1] introduced Attention-Based Knowledge Distillation (ABKD) to compress large GNNs into smaller ones while maintaining accuracy. Wu et al. [46] developed an approach focused on model extraction

attacks on GNNs, demonstrating effective duplication of models with high input-output correlation.

Latent feature distillation involves transferring intermediate representations from the teacher to the student model. Chen et al. [4] proposed a structure-aware MLP student and a structure-mixing distillation strategy to distill knowledge from GNNs into MLPs. Samy et al. [37] introduced Graph2Feat that enables inductive link prediction in graph learning through knowledge distillation, showing superior performance in terms of AUC and average precision. Joshi et al. [23] introduced graph contrastive representation distillation (G-CRD), aligning student node embeddings with those of the teacher in a shared representation space to preserve global topology.

Graph structure knowledge distillation focuses on transferring structural information from the teacher to the student model. Wu et al.’s Prototype-Guided Knowledge Distillation (PGKD) [50] method distills graph structural information from GNNs to MLPs without requiring graph edges. The graph-free knowledge distillation (GFKD) approach by Deng et al. [8] models graph topology structures for knowledge transfer without using graph data. Yang et al. [53] proposed VQGraph, a framework that transfers structural knowledge from GNNs to MLPs, achieving state-of-the-art performance in GNN-MLP distillation.

All these exceptional works offer valuable references and a robust theoretical foundation for the proposed LinguGKD framework in this paper.

6 CONCLUSION

In this study, we propose a novel LLM-to-GNN knowledge distillation framework termed LinguGKD, which integrates the semantic understanding capabilities of LLMs with the efficiency and structural insights of GNNs. LinguGKD employs TAG-oriented instruction tuning to train pre-trained LLMs as teacher models and introduces a layer-adaptive contrastive distillation strategy to align and transfer node features between teacher LLMs and student GNNs within a latent space. Extensive experiments across various LLM and GNN architectures on multiple datasets demonstrates that LinguGKD significantly enhances the predictive accuracy and convergence rate of GNNs without requiring additional training data or model parameters, making them highly practical for deployment in resource-constrained environments. Moreover, LinguGKD shows great potential for leveraging advancements in LLM research to continuously augment GNN performance.

REFERENCES

- [1] Anshul Ahluwalia, Rohit Das, Payman Behnam, Alind Khare, Pan Li, and Alexey Tumanov. 2023. ABKD: Graph Neural Network Compression With Attention-Based Knowledge Distillation. *arXiv Preprint arXiv:2310.15938* (2023).
- [2] AI@Meta. 2024. Llama 3 Model Card. (2024). https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md
- [3] Deyu Bo, Xiao Wang, Chuan Shi, and Huawei Shen. 2021. Beyond Low-Frequency Information in Graph Convolutional Networks. In *AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 35. 3950–3957.
- [4] Jie Chen, Shouzen Chen, Mingyuan Bai, Junbin Gao, Junping Zhang, and Jian Pu. 2022. SA-MLP: Distilling Graph Knowledge From GNNs Into Structure-Aware MLP. *arXiv Preprint arXiv:2210.09609* (2022).
- [5] Ming Chen, Zhewei Wei, Zengfeng Huang, Bolin Ding, and Yaliang Li. 2020. Simple and Deep Graph Convolutional Networks. In *International Conference on Machine Learning (ICML)*. 1725–1735.
- [6] Zhikai Chen, Haitao Mao, Hang Li, Wei Jin, Hongzhi Wen, Xiaochi Wei, Shuaiqiang Wang, Dawei Yin, Wenqi Fan, and Hui et al. Liu. 2024. Exploring The Potential Of Large Language Models (LLMs) In Learning On Graphs. *ACM SIGKDD Explorations Newsletter* 25, 2 (2024), 42–61.
- [7] Eli Chien, Wei Cheng Chang, Cho Jui Hsieh, Hsiang Fu Yu, Jiong Zhang, Olgica Milenkovic, and Inderjit S Dhillon. 2022. Node Feature Extraction By Self-Supervised Multi-Scale Neighborhood Prediction. In *International Conference on Learning Representations (ICLR)*.
- [8] Xiang Deng and Zhongfei Zhang. 2021. Graph-Free Knowledge Distillation for Graph Neural Networks. In *International Joint Conference on Artificial Intelligence (IJCAI)*. 2318–2324.
- [9] Tu Anh Dinh, Jeroen den Boef, Joran Cornelisse, and Paul Groth. 2023. E2EG: End-to-End Node Classification Using Graph Topology and Text-Based Node Attributes. In *IEEE International Conference on Data Mining Workshops (ICDMW)*. 1084–1091.
- [10] Keyu Duan, Qian Liu, Tat-Seng Chua, Shuicheng Yan, Wei Tsang Ooi, Qizhe Xie, and Junxian He. 2023. Simteg: A Frustratingly Simple Approach Improves Textual Graph Learning. *arXiv Preprint arXiv* (2023).
- [11] Vijay Prakash Dwivedi and Xavier Bresson. 2020. A Generalization of Transformer Networks to Graphs. *arXiv preprint arXiv:2012.09699* (2020).
- [12] Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2023. Talk Like A Graph: Encoding Graphs For Large Language Models. In *International Conference on Learning Representations (ICLR)*.
- [13] Zihao Fu, Wai Lam, Anthony Man-Cho So, and Bei Shi. 2021. A Theoretical Analysis of the Repetition Problem in Text Generation. In *AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 35. 12848–12856.
- [14] Will Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [15] Mingguo He, Zhewei Wei, Hongteng Xu, and Others. 2021. Bernnet: Learning Arbitrary Graph Spectral Filters via Bernstein Approximation. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 14239–14251.
- [16] Xiaoxin He, Xavier Bresson, Thomas Laurent, Perold, and Bryan Hooi. 2024. Harnessing Explanations: LLM-to-LM Interpreter for Enhanced Text-Attributed Graph Representation Learning. In *International Conference on Learning Representations (ICLR)*.
- [17] Yufei He and Yao Ma. 2022. SGKD: A Scalable and Effective Knowledge Distillation Framework for Graph Representation Learning. In *IEEE International Conference on Data Mining Workshops (ICDMW)*. 666–673.
- [18] Or Honovich, Thomas Scialom, Omer Levy, and Timo Schick. 2023. Unnatural Instructions: Tuning Language Models with (Almost) No Human Labor. In *Annual Meeting of the Association for Computational Linguistics (ACL)*. 14409–14428.
- [19] Weihua Hu, Matthias Fey, Marinka Zitnik, Yuxiao Dong, Hongyu Ren, Bowen Liu, Michele Catasta, and Jure Leskovec. 2020. Open Graph Benchmark: Datasets for Machine Learning on Graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33. 22118–22133.
- [20] Ling Huang, Chang-Dong Wang, and Hong-Yang Chao. 2019. Higher-Order Multi-Layer Community Detection. In *The AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 33. 9945–9946.
- [21] Xuanwen Huang, Kaiqiao Han, Dezheng Bao, Quanjin Tao, Zhisheng Zhang, Yang Yang, and Qi Zhu. 2023. Prompt-Based Node Feature Extractor for Few-Shot Learning on Text-Attributed Graphs. *arXiv Preprint arXiv:2309.02848* (2023).
- [22] Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, and Lucile et al. Saulnier. 2023. Mistral 7B. *arXiv Preprint arXiv:2310.06825* (2023).
- [23] Chaitanya K. Joshi, Fayao Liu, Xu Xun, Jie Lin, and Chuan Sheng Foo. 2024. On Representation Knowledge Distillation for Graph Neural Networks. *IEEE Transactions on Neural Networks and Learning Systems* 35, 4 (2024), 4656–4667.
- [24] Thomas N. Kipf and Max Welling. 2017. Semi-Supervised Classification with Graph Convolutional Networks. In *International Conference on Learning Representations (ICLR)*.
- [25] Guohao Li, Matthias Müller, Bernard Ghanem, and Vladlen Koltun. 2021. Training Graph Neural Networks with 1000 Layers. In *International Conference on Machine Learning (ICML)*. 6437–6449.
- [26] Guohao Li, Chenxin Xiong, Ali Thabet, and Bernard Ghanem. 2020. DeeperGCN: All You Need to Train Deeper GCNs. *arXiv preprint arXiv:2006.07739* (2020).
- [27] Jianxin Li, Hao Peng, Yuwei Cao, Yingdong Dou, Hekai Zhang, Philip S. Yu, and Lifang He. 2023. Higher-Order Attribute-Enhancing Heterogeneous Graph Neural Networks. *IEEE Transactions on Knowledge and Data Engineering* 35, 1 (2023), 560–574.
- [28] Yuhao Li, Zhixun Li, Peisong Wang, Jia Li, Xiangguo Sun, Hong Cheng, and Jeffrey Xu Yu. 2023. A Survey of Graph Meets Large Language Model: Progress and Future Directions. *arXiv Preprint arXiv:2311.12399* (2023).
- [29] Yanying Li, Xiuling Wang, Yue Ning, and Hui Wang. 2022. Fairlp: Towards Fair Link Prediction on Social Network Graphs. In *International AAAI Conference on Web and Social Media (ICWSM)*, Vol. 16. 628–639.
- [30] Yihao Li, Ru Zhang, Jianyi Liu, and Gongshen Liu. 2024. An Enhanced Prompt-Based LLM Reasoning Scheme via Knowledge Graph-Integrated Collaboration. *arXiv Preprint arXiv:2402.04978* (2024).
- [31] Sitao Luan, Chenqing Hua, Qincheng Lu, Jiaqi Zhu, Mingde Zhao, Shuyuan Zhang, Xiao-Wen Chang, and Doina Precup. 2022. Revisiting Heterophily for Graph Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 1362–1375.
- [32] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation Learning with Contrastive Predictive Coding. *arXiv preprint arXiv:1807.03748* (2018).
- [33] OpenAI. 2023. Prompt Engineering. <https://platform.openai.com/docs/guides/prompt-engineering>. Accessed: 2023-12-20.
- [34] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Leike Jan, and Lowe Ryan. 2022. Training Language Models to Follow Instructions with Human Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 35. 27730–27744.
- [35] Shirui Pan, Linhao Luo, Yufei Wang, Chen Chen, Jiapu Wang, and Xindong Wu. 2024. Unifying Large Language Models and Knowledge Graphs: A Roadmap. *IEEE Transactions on Knowledge and Data Engineering* (2024).
- [36] Chen Qian, Huayi Tang, Zhirui Yang, Hong Liang, and Yong Liu. 2023. Can Large Language Models Empower Molecular Property Prediction? *arXiv Preprint arXiv:2307.07443* (2023).
- [37] Ahmed E. Samy, Zekarias T. Kefato, and Sarunas Girdzijauskas. 2023. Graph2Feat: Inductive Link Prediction via Knowledge Distillation. In *ACM Web Conference (WWW)*. 805–812.
- [38] Yunsheng Shi, Zhengjie Huang, Shikun Feng, Hui Zhong, Wenjin Wang, and Yu Sun. 2020. Masked Label Prediction: Unified Message Passing Model for Semi-Supervised Classification. *arXiv preprint arXiv:2009.03509* (2020).
- [39] Qingyi Si, Tong Wang, Zheng Lin, Xu Zhang, Yanan Cao, and Weiping Wang. 2023. An Empirical Study of Instruction-Tuning Large Language Models in Chinese. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
- [40] Yanchao Tan, Zihao Zhou, Hang Lv, Weiming Liu, and Carl Yang. 2023. WalkLM: A Uniform Language Model Fine-Tuning Framework for Attributed Graph Embedding. In *Conference on Neural Information Processing Systems (NeurIPS)*.
- [41] Hugo Touvron, Thibaut Lavril, Gautier Lacroix, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, and Faisal Azhar. 2023. Llama: Open And Efficient Foundation Language Models. *arXiv Preprint arXiv:2302.13971* (2023).
- [42] Ashish Vaswani, Noam Shazeer, Niki Parmar, Uszkoireit, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 30.
- [43] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks. In *International Conference on Learning Representations (ICLR)*.
- [44] Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2024. Can Language Models Solve Graph Problems in Natural Language?. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.
- [45] Wei Wei, Xubin Ren, Jiabin Tang, Qinyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Llmrec: Large Language Models with Graph Augmentation for Recommendation. In *ACM International Conference on Web Search and Data Mining (WSDM)*. 806–815.
- [46] Bang Wu, Xiangwen Yang, Shirui Pan, and Xingliang Yuan. 2022. Model Extraction Attacks on Graph Neural Networks: Taxonomy and Realisation. In *Asia Conference on Computer and Communications Security (ASIA-CCS)*. 337–350.
- [47] Felix Wu, Amauri Souza, Tianyi Zhang, Christopher Fifty, Tao Yu, and Kilian Weinberger. 2019. Simplifying Graph Convolutional Networks. In *International*

- Conference on Machine Learning (ICML)*. 6861–6871.
- [48] Nan Wu and Chaofan Wang. 2022. Gtnet: A Tree-based Deep Graph Learning Architecture. *arXiv preprint arXiv:2204.12802* (2022).
 - [49] Qitian Wu, Wentao Zhao, Chenxiao Yang, Hengrui Zhang, Fan Nie, Haitian Jiang, Yatao Bian, and Junchi Yan. 2024. Simplifying and Empowering Transformers for Large-Graph Representations. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 36.
 - [50] Taiqiang Wu, Zhe Zhao, Jiahao Wang, Xingyu Bai, Lei Wang, Ngai Wong, and Yujiu Yang. 2023. Edge-Free But Structure-Aware: Prototype-Guided Knowledge Distillation From GNNs To MLPs. *arXiv Preprint arXiv:2303.13763* (2023).
 - [51] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighof. 2023. C-Pack: Packaged Resources to Advance General Chinese Embedding. *arXiv Preprint arXiv:2309.07597* (2023).
 - [52] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. 2018. How Powerful Are Graph Neural Networks? *arXiv Preprint arXiv:1810.00826* (2018).
 - [53] Ling Yang, Ye Tian, Minkai Xu, Zhongyi Liu, Shenda Hong, Wei Qu, Wentao Zhang, Bin Cui, Muhan Zhang, and Jure Leskovec. 2024. VQGraph: Rethinking Graph Representation Space for Bridging GNNs and MLPs. In *International Conference on Learning Representations (ICLR)*.
 - [54] Renchi Yang and Jieming Shi. 2024. Efficient High-Quality Clustering for Large Bipartite Graphs. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–27.
 - [55] Zhilin Yang, William Cohen, and Ruslan Salakhudinov. 2016. Revisiting Semi-Supervised Learning with Graph Embeddings. In *International Conference on Machine Learning (ICML)*, Vol. 48. 40–48.
 - [56] Ruosong Ye, Caiqi Zhang, Runhui Wang, Shuyuan Xu, and Yongfeng Zhang. 2023. Natural Language Is All A Graph Needs. *arXiv Preprint arXiv:2308.07134* (2023).
 - [57] Chengxuan Ying, Tianle Cai, Shengjie Luo, Shuxin Zheng, Guolin Ke, Di He, Yanming Shen, and Tie-Yan Liu. 2021. Do Transformers Really Perform Badly for Graph Representation?. In *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 34. 28877–28888.
 - [58] Jiong Zhang, Wei-Cheng Chang, Hsiang-Fu Yu, and Inderjit Dhillon. 2021. Fast Multi-Resolution Transformer Fine-Tuning For Extreme Multi-Label Text Classification. In *Advances In Neural Information Processing Systems (NeurIPS)*, Vol. 34. 7267–7280.
 - [59] Shengyu Zhang, Linfeng Dong, Xiaoya Li, Sen Zhang, Xiaofei Sun, Shuhe Wang, Jiwei Li, Runyi Hu, Tianwei Zhang, Fei Wu, et al. 2023. Instruction Tuning for Large Language Models: A Survey. *arXiv preprint arXiv:2308.10792* (2023).
 - [60] Xuanyu Zhang. 2020. Cfgnn: Cross Flow Graph Neural Networks for Question Answering on Complex Tables. In *AAAI Conference on Artificial Intelligence (AAAI)*, Vol. 34. 9596–9603.
 - [61] Jing Zhu, Xiang Song, Vassilis N Ioannidis, Danai Koutra, and Christos Faloutsos. 2023. TouchUp-G: Improving Feature Representation Through Graph-Centric Finetuning. *arXiv Preprint arXiv:2309.13885* (2023).